



Docker Handbook

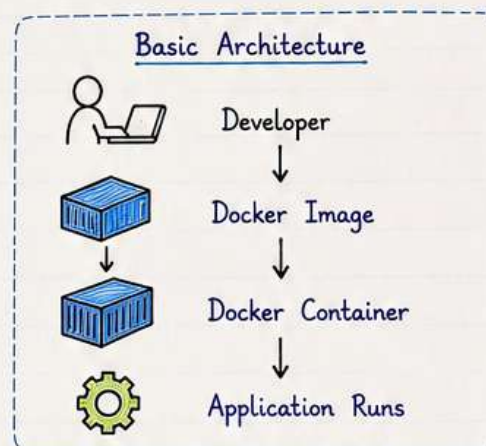
Your Path to
Containerization 

1. What is Docker?

Docker is a **containerization platform** that packages an application and all its dependencies (libraries, runtime, tools, configurations, etc.) into a lightweight, portable container. It ensures that the application runs consistently across different environments.

2. Why Docker?

- "Works on my machine" issue
- Dependency conflicts between projects
- Different operating systems and environments
- Complex and manual deployment process
- Difficult to scale applications
- Inconsistent configuration management



Docker solves these problems by packaging everything together and making the application portable, consistent and easy to deploy.

3. Features of Docker



Containerization
Isolated environment for applications.



Lightweight
Uses fewer resources compared to VMs.



Fast Startup
Containers start in seconds.



Portable
Run anywhere - laptop, server, cloud.



Isolated
Each container runs in its own space.



Version Control
Images can be versioned and shared.



Easy Deployment
Build once, run anywhere.



Scalability
Easily scale apps up or down.



Consistency
Same behavior across all environments.

4. Virtual Machine vs Docker

Aspect	Virtual Machine	Docker Container
Includes	Guest OS	Shares Host OS Kernel
Size	Heavy and bulky	Lightweight
Startup Time	Slow (Minutes)	Fast (Seconds)
Resource Usage	Consumes more memory	Consumes less memory
Portability	Harder to move and scale	Easy to move and scale
Virtualization	Hardware Virtualization	OS-level Virtualization

5. How Docker Works?

1. You create a **Docker Image** using a Dockerfile.
↓
2. **Docker Image** contains the application and all its dependencies.
↓
3. When you run the image, Docker creates a **Container** from it.
↓
4. The container runs the application in an **isolated environment**.

From My Work

Formula ERP System

- Stack: ASP.NET Core, C#, SQL Server, React
- ✓ Multi-tenant ERP for Formula Group
 - ✓ Dockerized microservices for Modules
 - ✓ Improved deployment speed & scalability

DotNet Developer
@ Formula Group



Lead Management System

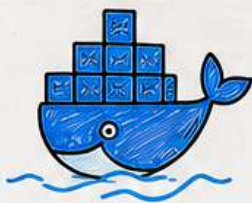
- Stack: ASP.NET Core, C#, SQL Server, React
- ✓ Lead capture, tracking & analytics
 - ✓ Containerized API & Background Jobs
 - ✓ Seamless CI/CD with Docker & GitHub Actions



Interview Tip

Docker packages an application and its dependencies into a **portable container** that runs consistently across environments.

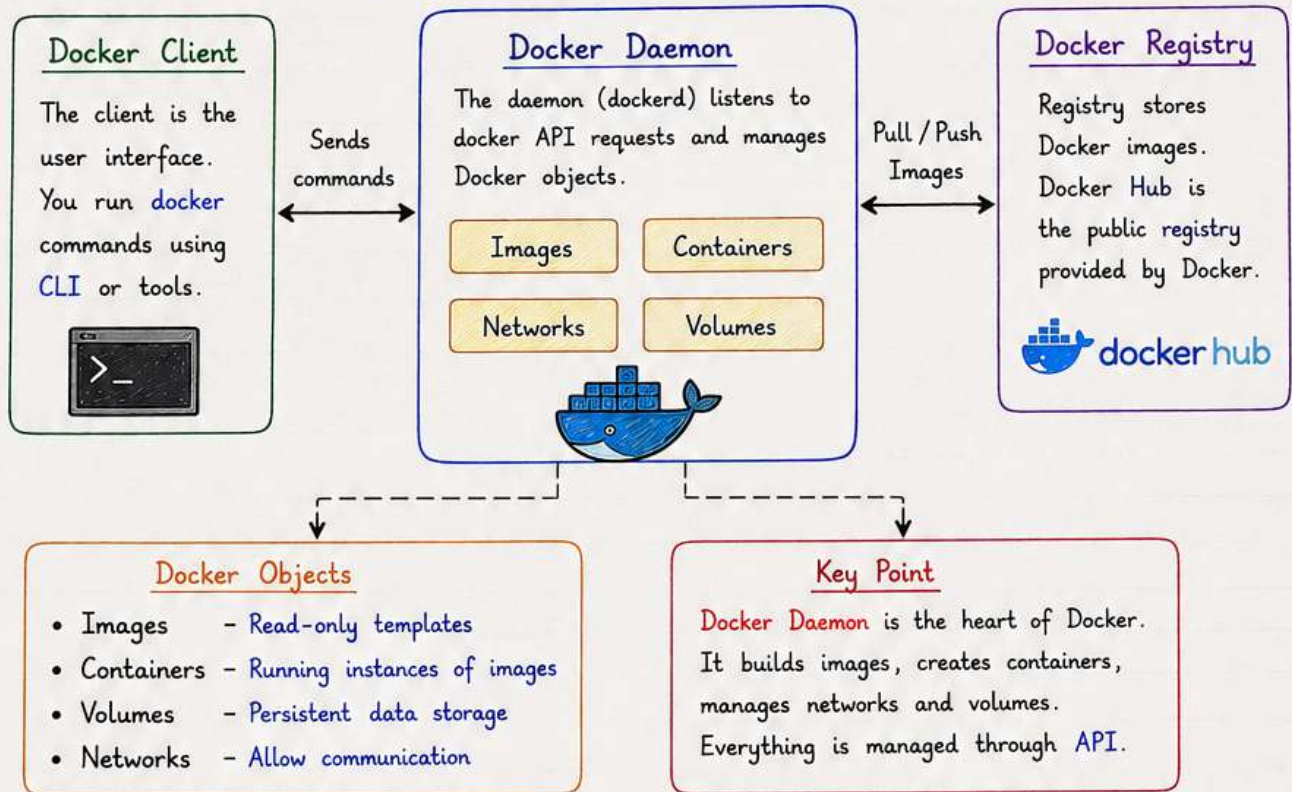




Docker Handbook

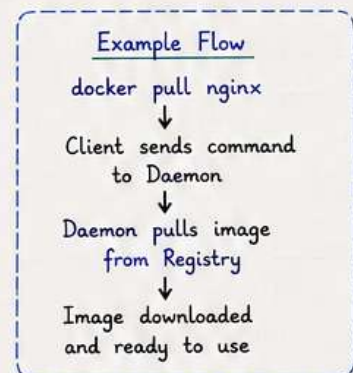
2. Docker Architecture

Docker follows a **client-server architecture**. Different components work together to **build, run and manage containers**.



★ How Components Communicate?

- The **Docker Client** sends commands to the **Docker Daemon**.
- The **Daemon** processes these commands and manages Docker objects.
- The **Daemon** pulls or pushes images to/from **Docker Registry** if required.
- All communication happens through **REST API** over Unix socket or TCP.



★ Key Components Explained

- ① **Docker Client** - You interact using **CLI** (docker commands) or tools.
- ② **Docker Daemon** - Background service that builds, runs and manages containers.
- ③ **Docker Registry** - Stores and distributes Docker images. (Docker Hub, private registry)
- ④ **Docker Objects** - Images, Containers, Volumes, Networks are the core building blocks.



Interview Tip

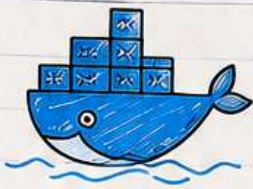
Docker uses a **client-server architecture**. The **Docker Daemon** is the engine that does the heavy lifting.



Remember:

Without Docker Daemon, Docker Client cannot perform any action. The Daemon is responsible for all Docker operations.





@codewithZ Docker Handbook

Page 3

3. Images & Containers

What is an Image?

A Docker Image is a lightweight, standalone, executable package that includes everything needed to run an application - code, runtime, libraries, tools, environment variables, config files, etc.

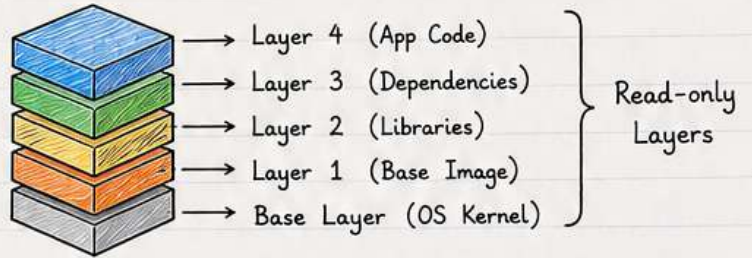
What is a Container?

A Container is a running instance of an image. It contains everything needed to run the application and is isolated from other containers and the host system.

Image Layers

Docker images are built in layers.

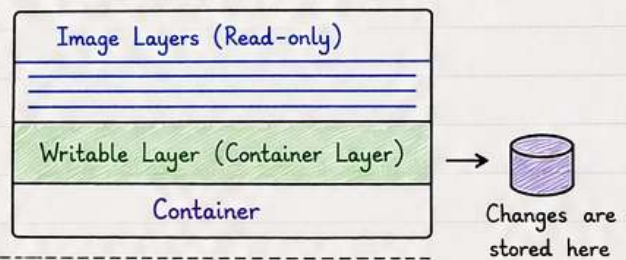
- Each instruction in Dockerfile creates a layer.
- Layers are read-only.
- Layers are cached and reused.
- This makes images small and fast.



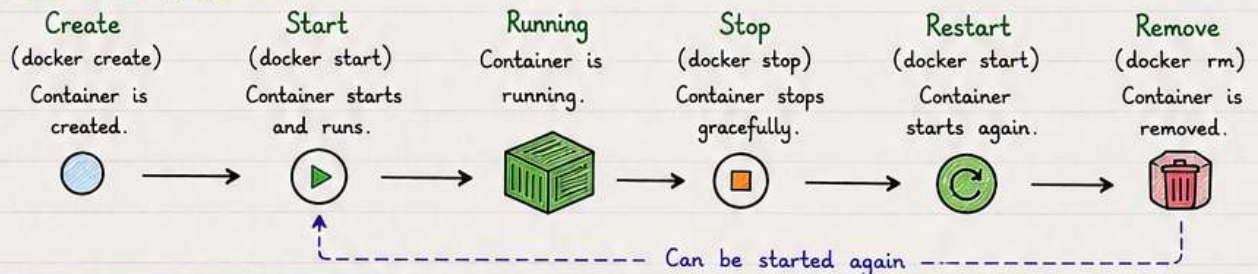
Container Layer (Writable Layer)

When a container runs, a thin writable layer is added on top of the image layers.

- All changes in the container are stored here.



Container Lifecycle



Important Docker Commands

Command	Description	Example
<code>docker pull <image></code>	Download an image from Docker Hub	<code>docker pull nginx</code>
<code>docker images</code>	List all downloaded images	<code>docker images</code>
<code>docker run <image></code>	Create and start a container	<code>docker run -d -p 8080:80 nginx</code>
<code>docker ps</code>	List running containers	<code>docker ps</code>
<code>docker ps -a</code>	List all containers (including stopped)	<code>docker ps -a</code>
<code>docker stop <container_id></code>	Stop a running container	<code>docker stop a1b2c3d4</code>
<code>docker start <container_id></code>	Start a stopped container	<code>docker start a1b2c3d4</code>
<code>docker rm <container_id></code>	Remove a container	<code>docker rm a1b2c3d4</code>
<code>docker rmi <image_id></code>	Remove an image	<code>docker rmi nginx</code>
<code>docker logs <container_id></code>	View container logs	<code>docker logs a1b2c3d4</code>
<code>docker exec -it <id> sh</code>	Access container terminal	<code>docker exec -it a1b2c3d4 sh</code>



Interview Tip

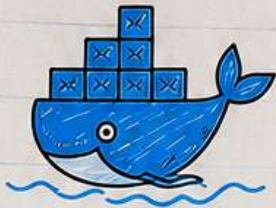
An Image is a blueprint, a Container is a running instance of that blueprint.



Remember

Images are read-only.
Containers are ephemeral.
Use volumes for persistent data.





Docker Handbook

4. Dockerfile

A Dockerfile is a text file that contains instructions to build a Docker image.

Dockerfile Instructions

- **FROM** : Specifies the base image to start from.
- **WORKDIR** : Sets the working directory inside the container.
- **COPY** : Copies files or directories from host to container.
- **ADD** : Similar to COPY, but can also extract archives and download files from URLs.
- **RUN** : Executes commands during image build.
- **EXPOSE** : Informs Docker that the container listens on the specified port(s).
- **ENV** : Sets environment variables inside the container.
- **CMD** : Provides default command to run when container starts.
- **ENTRYPOINT** : Configures a container that will run as an executable.

Example Dockerfile

```
# Use official OpenJDK image
FROM openjdk:21-jdk-slim
WORKDIR /app
COPY app.jar /app/app.jar
EXPOSE 8080
ENV APP_NAME=DockerDemo
ENTRYPOINT ["java", "-jar", "app.jar"]
```

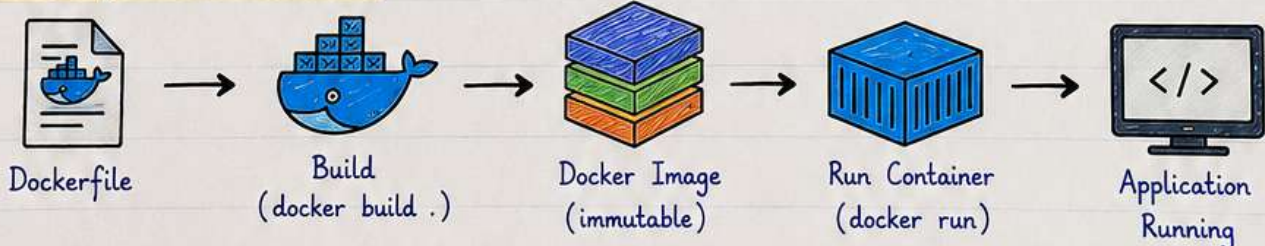


Dockerfile is used to automate the process of creating Docker images in a repeatable and versionable way.

Example Explanation

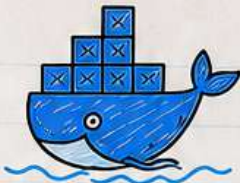
- **FROM openjdk:21-jdk-slim** → Uses official OpenJDK 21 slim image as base.
- **WORKDIR /app** → Sets working directory to /app.
- **COPY app.jar /app/app.jar** → Copies app.jar from host to /app directory.
- **EXPOSE 8080** → Document that container listens on port 8080.
- **ENV APP_NAME=DockerDemo** → Sets environment variable APP_NAME.
- **ENTRYPOINT ["java", "-jar", "app.jar"]** → Runs the Java application when container starts.

Dockerfile Build Process



Important Notes

- Each instruction in a Dockerfile creates a new layer in the image.
- Order of instructions matters. Put less frequently changed instructions first for better caching.
- Use `.dockerignore` file to exclude unnecessary files from the build context.
- Keep images small by using slim base images and multi-stage builds.



Docker Handbook

5. Docker Volumes & Bind Mounts

Why do we need Volumes?

- Containers are ephemeral – when a container is removed, all data inside it is lost.
- Volumes provide **persistent storage** that survives container restarts and removals.
- Useful for **databases, logs, uploads, and configuration files**.
- Keeps data separate from container lifecycle.

Types of Volumes

- ① **Named Volumes** – Managed by Docker and stored in Docker host.
- ② **Anonymous Volumes** – Created by Docker automatically, not given a name.
- ③ **Bind Mounts** – Map a file or directory from host machine to container.

1) Named Volume

- Created and managed by Docker.
- Stored in `/var/lib/docker/volumes/`
- Recommended for production.
- Easy to back up and migrate.

Example:

```
docker run -d -v mydata:/app/data nginx
```

2) Anonymous Volume

- Docker creates a volume automatically.
- No specific name is assigned.
- Useful for quick and temporary tasks.
- Deleted when container is removed.

Example:

```
docker run -d -v /app/data nginx
```

3) Bind Mount

- Map a path from host to container.
- Any changes reflect on host.
- Useful for development.
- Host path must exist.

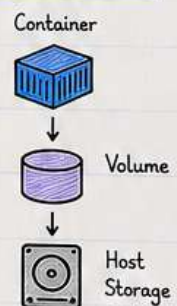
Example:

```
docker run -d -v /home/user/data:/app/data nginx
```

Volume vs Bind Mount

Feature	Volume	Bind Mount
Managed by	Docker	User (Host)
Location	Inside Docker storage (<code>/var/lib/docker/volumes/</code>)	Any path on Host system
Portability	High	Low (depends on host path)
Performance	Better (Docker optimized)	Slightly slower
Security	More secure	Less secure (direct host access)
Use Case	Production, persistent data	Development, config, source code

How volumes work



Common Commands

- `docker volume ls` → List all volumes
- `docker volume create myvol` → Create a named volume
- `docker run -v myvol:/app/data nginx` → Run container with named volume
- `docker run -v /host/path:/app nginx` → Run container with bind mount
- `docker volume inspect myvol` → Inspect volume details
- `docker volume rm myvol` → Remove a volume
- `docker volume prune` → Remove unused volumes



Best Practices

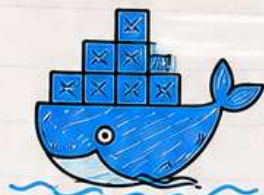
- ✓ Use volumes for data that needs to persist.
- ✓ Use bind mounts only for development.
- ✓ Never store application data inside the container.
- ✓ Backup important volumes regularly.
- ✓ Keep volumes small and organized.



Interview Tip

Containers are disposable, but data is valuable. Volumes help us persist data independently of container lifecycle.





Docker Handbook

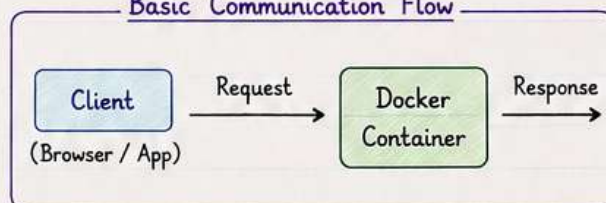
6. Docker Networking

Docker networking allows containers to communicate with each other, with external systems and with the internet.

Why Networking?

- Containers are **isolated** by default.
- Networking enables communication between containers, host and outside world.
- It provides **flexibility**, **security** and **scalability**.

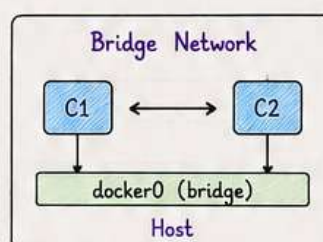
Basic Communication Flow



Types of Docker Networks

① Bridge (default)

- Default network for containers.
- Containers can communicate with each other.
- External access through port mapping.

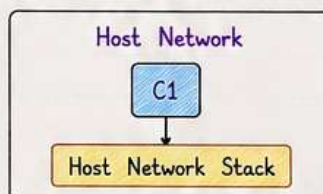


Example Command

```
docker network create my-bridge
```

② Host Network

- Container **shares** the host's network stack.
- No isolation with host.
- Better performance, but less secure.

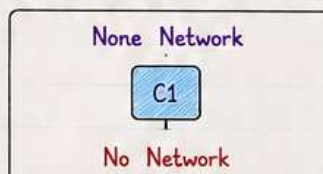


Example Command

```
docker run --network host nginx
```

③ None Network

- Container has **no** networking.
- Completely isolated.
- Useful for security.

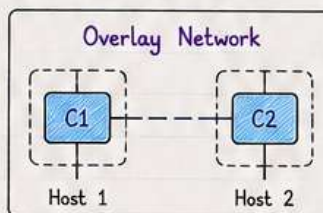


Example Command

```
docker run --network none nginx
```

④ Overlay Network (for multiple hosts)

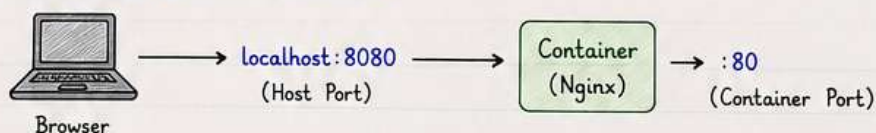
- Connects containers across **multiple** Docker hosts.
- Used in Docker Swarm and Kubernetes.
- Enables cross-host communication.



Example Command

```
docker network create -d overlay my-overlay
```

Port Mapping (Publish Ports)



Command Example

```
docker run -d -p 8080:80 nginx
```

↓ ↓
Host Port Container Port

Important Points

- ★ By default, containers can communicate with each other on the same **bridge network**.
- ★ To access a container from outside, use **-p** (publish) option.
- ★ Use **user-defined networks** for better control and isolation.
- ★ Avoid **host network** in production unless necessary.



Interview Tip

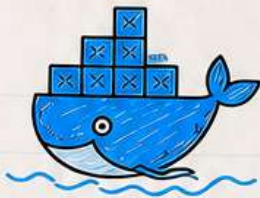
Docker networking gives containers isolation, connectivity and flexibility to communicate securely.



Remember

Default network = **bridge**
Fastest network = **host**

Most isolated = **none**
Cross-host communication = **overlay**



Docker Handbook

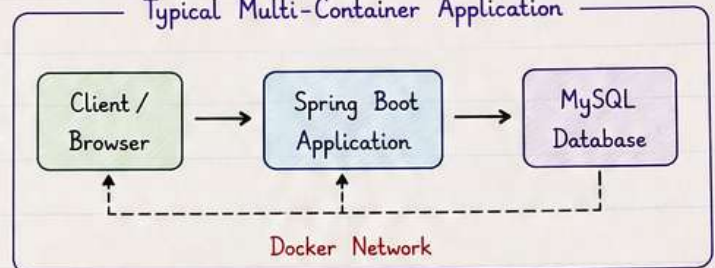
7. Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications. It uses a YAML file (`docker-compose.yml`) to configure application services.

Why Compose?

- Define multi-container applications.
- Manage complex configurations easily.
- Start, stop and scale services with one command.
- Handle networks and volumes automatically.
- Great for development and testing.

Typical Multi-Container Application



`docker-compose.yml`

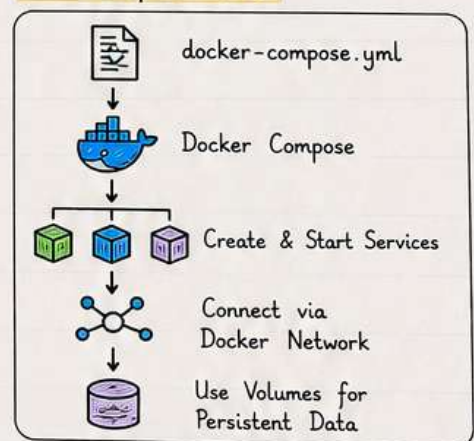
```
version: '3.8'
services:
  app:
    build: .
    container_name: spring-boot-app
    ports:
      - "8080:8080"
    environment:
      - SPRING_DATASOURCE_URL=jdbc:mysql://db:3306/mydb
      - SPRING_DATASOURCE_USERNAME=root
      - SPRING_DATASOURCE_PASSWORD=root
    depends_on:
      - db
    networks:
      - app-network
  db:
    image: mysql:8.0
    container_name: mysql-db
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=root
      - MYSQL_DATABASE=mydb
    volumes:
      - db-data:/var/lib/mysql
    networks:
      - app-network
volumes:
  db-data:
networks:
  app-network:
    driver: bridge
```

File format version
Application service
Build image from Dockerfile in current directory
Map port 8080 on host to 8080 in container
Environment variables
Start db service first
Connect to custom network
Database service
Use official MySQL image
Auto restart container
DB credentials & name
Persist database data
Connect to custom network
Named volume for DB data
Custom network for services
Use bridge network driver

Common Compose Commands

Command	Description
<code>docker-compose up</code>	Build and start containers
<code>docker-compose up -d</code>	Start in detached mode
<code>docker-compose down</code>	Stop and remove containers, networks, volumes
<code>docker-compose ps</code>	List running services
<code>docker-compose logs</code>	View logs of services
<code>docker-compose build</code>	Build or rebuild services
<code>docker-compose restart</code>	Restart all services
<code>docker-compose exec <svc> <cmd></code>	Run command in a service

How Compose Works



Benefits

- ✓ Simplifies multi-container application management.
- ✓ Ensures containers communicate using service names.
- ✓ Manages networks, volumes, and dependencies.
- ✓ Easy to version control and share.
- ✓ Speeds up local development workflow.

Key Points

- ★ Service names in compose act as hostnames.
- ★ All services in same network can talk to each other.
- ★ Volumes ensure data persists even if containers are removed.
- ★ `depends_on` does not wait for service to be ready, only for start.



Interview Tip

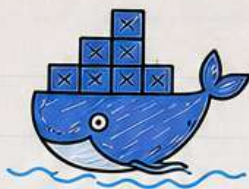
Docker Compose is best for local development and small deployments. For production scale, use Kubernetes.



Remember

- Compose file defines services, networks and volumes.
- One command can start an entire application stack.
- Keep compose file clean, readable and modular.





Docker Handbook

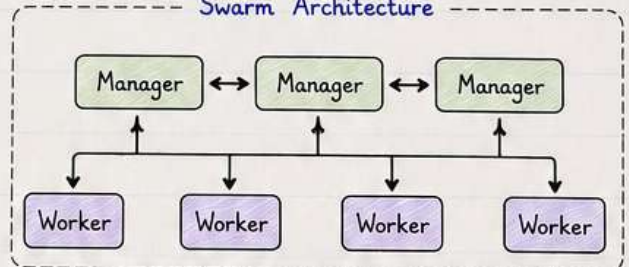
8. Docker Swarm

Docker Swarm is Docker's native clustering and orchestration tool for managing a cluster of Docker nodes as a single virtual system.

Key Concepts

- **Node** - A machine in the swarm (Manager or Worker).
- **Manager** - Manages the swarm and makes decisions.
- **Worker** - Executes tasks assigned by managers.
- **Service** - Defines the desired state of containers.
- **Task** - A running instance of a service.
- **Stack** - Group of related services (using Compose file).

Swarm Architecture



Swarm Mode Commands

Command	Description
<code>docker swarm init</code>	Initialize a swarm (on manager node)
<code>docker swarm join</code>	Join a node to the swarm
<code>docker node ls</code>	List all nodes in the swarm
<code>docker node inspect <node_id></code>	Inspect a node
<code>docker node rm <node_id></code>	Remove a node from swarm
<code>docker swarm leave</code>	Leave the swarm
<code>docker swarm update</code>	Update swarm configuration

Service Commands

Command	Description
<code>docker service create</code>	Create a new service
<code>docker service ls</code>	List all services
<code>docker service ps <service></code>	List tasks of a service
<code>docker service inspect <service></code>	Inspect a service
<code>docker service scale <service>=<n></code>	Scale a service
<code>docker service update <service></code>	Update a service
<code>docker service rm <service></code>	Remove a service

Create a Swarm

1. Initialize Swarm

```
$ docker swarm init
```

(Manager Node)

2. Join Worker Node

```
$ docker swarm join  
--token <token>
```

(Worker Node)

3. Create Service

```
$ docker service create  
--replicas 3 nginx
```

(Manager Node)

4. Service Running

nginx service

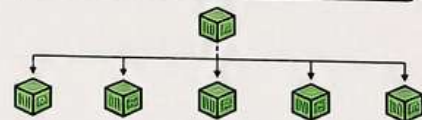
Example: Create a Service

```
$ docker service create \  
--name web \  
--replicas 3 \  
--publish 80:80 \  
nginx
```

--name Service name
--replicas Number of task replicas
--publish Publish port (host:container)
nginx — Image

Scaling a Service

```
$ docker service scale web=5
```



Stack (Using Compose File)

1. Create a docker-compose.yml file.
2. Deploy the stack:

```
$ docker stack deploy -c docker-compose.yml myapp
```

3. List stacks:

```
$ docker stack ls
```

4. Remove stack:

```
$ docker stack rm myapp
```

Example docker-compose.yml

```
version: '3.8'  
services:  
  web:  
    image: nginx  
    ports:  
      - "80:80"  
    deploy:  
      replicas: 3
```

Benefits of Docker Swarm

- ✓ Native to Docker - no extra tools.
- ✓ Simple to set up.
- ✓ High availability and scalability.
- ✓ Rolling updates and rollback.
- ✓ Load balancing out of the box.
- ✓ Works across multiple environments.



Interview Tip

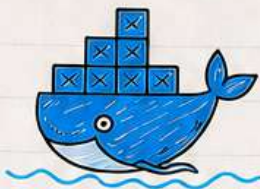
Docker Swarm is great for simple and small-to-medium clusters. For advanced orchestration, use Kubernetes.



Remember

A swarm turns a group of Docker hosts into a single, unified platform.





Docker Handbook

9. Docker Images

A Docker image is a read-only template used to create containers. Images are built layer by layer.

Key Points

- Images are immutable.
- Each instruction in a Dockerfile creates a new layer.
- Layers are stacked to form the final image.
- Layers are shared between images to save space.
- Images are stored in image registry (Docker Hub / Private).

Image Layers

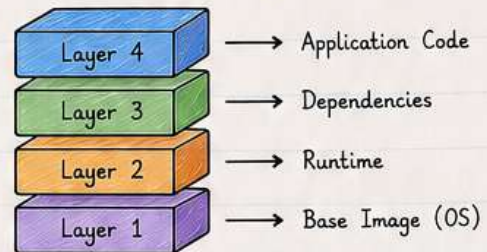
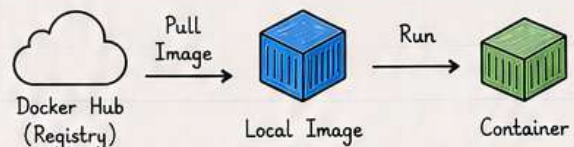


Image Commands

Command	Description
<code>docker images</code>	List all images
<code>docker images -a</code>	List all images (including intermediate)
<code>docker pull <image></code>	Pull image from a registry
<code>docker rmi <image></code>	Remove an image
<code>docker rmi -f <image></code>	Force remove an image
<code>docker image prune</code>	Remove dangling images
<code>docker inspect <image></code>	Show detailed image information
<code>docker history <image></code>	Show layers and commands of image

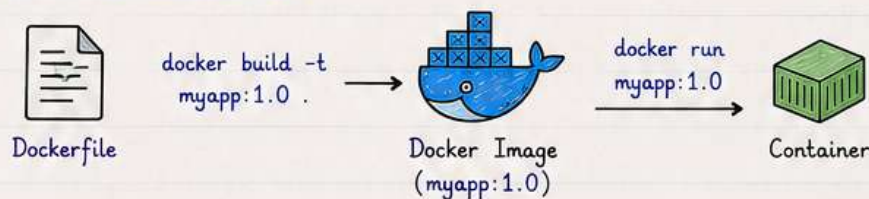
Image Pull & Run



Example

```
docker pull nginx:latest      # Pull image
docker images                 # List images
docker run -d -p 80:80 nginx  # Run container
```

Dockerfile to Image



Example Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "app.js"]
```

Image Tags

Images are usually tagged with a name and optional version.

- `myapp:latest` → latest version
- `myapp:1.0` → specific version
- `nginx:alpine` → alpine variant

Common Tags	Purpose
<code>latest</code>	Latest available version
<code>alpine</code>	Lightweight variant
<code>stable</code>	Stable release
<code><version></code>	Specific version (e.g., 1.0, 2.1.3)

★ Best Practices

- ✓ Use official or trusted base images.
- ✓ Keep images small (use alpine, multi-stage builds).
- ✓ Use `.dockerignore` to exclude unnecessary files.
- ✓ Scan images for vulnerabilities.
- ✓ Regularly update base images.



Interview Tip

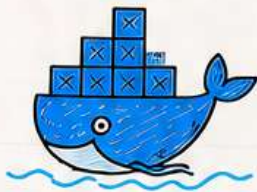
Images are like blueprints (read-only). Containers are running instances (read-write). A container can be created from the same image any number of times.



Remember

One image can create many containers.
Keep images small, secure and up to date!





Docker Handbook

10. Docker Volumes (Deep Dive)

Volumes provide **persistent** data that survives container restarts and removals. They are stored **outside** the container filesystem.

Why Volumes?

- Data persists beyond container lifecycle.
- Can be shared between containers.
- Better performance than bind mounts on some OS.
- Managed by Docker.
- Easy to backup and migrate.

Types of Volumes

1) Named Volume

- Managed by Docker.
- Stored in Docker host.
- Has a unique name.
- Portable across environments.

Example:

```
docker volume create mydata
```

2) Anonymous Volume

- Created automatically when not given a name.
- Useful for temporary data.
- Removed when container is removed.

Example:

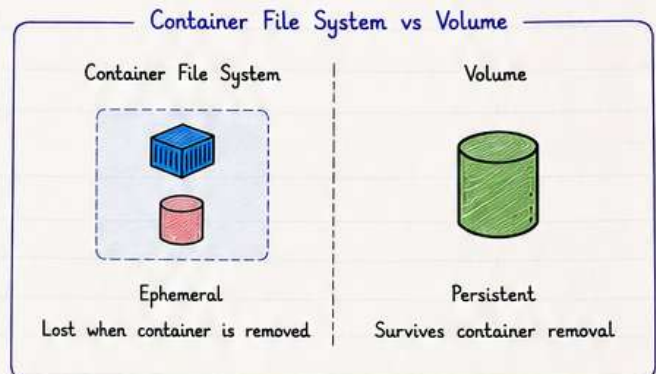
```
docker run -d -v /app/data nginx
```

3) Bind Mount

- Maps a file or directory from host.
- Direct access to host files.
- Useful for development.
- Host path must exist.

Example:

```
docker run -d -v /home/user/data:/data nginx
```



Volume Operations

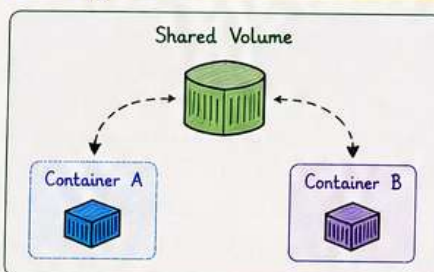
Command	Description	Example
docker volume ls	List all volumes	docker volume ls
docker volume inspect <name>	Inspect volume details	docker volume inspect mydata
docker volume create <name>	Create a named volume	docker volume create mydata
docker volume rm <name>	Remove a volume	docker volume rm mydata
docker volume prune	Remove unused volumes	docker volume prune

Where are Volumes Stored?



Linux: /var/lib/docker/volumes/
macOS: /var/lib/docker/volumes/
Windows: C:\ProgramData\docker\volumes\
(Managed by Docker engine)

Sharing Volumes Between Containers



Example

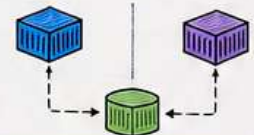
```
# Create volume
docker volume create shared-data

# Run first container
docker run -d --name app1 -v shared-data:/data nginx

# Run second container using same volume
docker run -d --name app2 -v shared-data:/data nginx
```

Check Inside Containers

Both containers share the same /data content.



★ Best Practices

- ✓ Use named volumes for persistent application data.
- ✓ Avoid bind mounts in production.
- ✓ Backup important volumes regularly.
- ✓ Keep volumes small and organized.
- ✓ Use volume labels for better management.



Interview Tip

Volumes are managed by Docker and live outside containers. Use volumes for persistent data and bind mounts for development convenience. Know the difference and use case for each type!



Remember

If the container is the process, the volume is the data it needs to keep. Containers come and go, but volumes stay!

